CloudSys Master's course

List of proposed projects

Presentation of projects: Tuesday 29th Sept. 2026

Final deadline for the selection of projects: Tuesday 6th Oct. 2026, 8h45

Start of the projects: Tuesday 6th Oct. 2026

Final deadline for submission of projects: 3rd Nov. 2026, 8h45.

The text in grey is optional. Only do this if you have time.

Project 1: Data cooperatives **2**

Project 2: Real-Time IoT Device Monitoring & Cloud Dashboarding Platform **6**

Project 3: Cloud-Native Trade Anomaly Detection & Reporting Platform **11**

Project 4: Self-adaptive ML prediction of energy consumption. **15**

Project 5: Monitoring Kubernetes clusters with AI **18**

Project 1: Data cooperatives

Proposed by some cantonal offices of Geneva Canton

Key contacts: Nabil Abdennadher, Francisco Mendonça, Raoul Dupuis

1. Context and Motivation

The Canton of Geneva employs several companies to operate several thousand IoT devices across its territory. Most currently transmit their data to servers outside Switzerland, creating a reliance on external providers, increasing operational costs, and limiting control over how public data is managed.

As data-protection requirements become stricter, and IoT hardware becomes more affordable, the canton aims to take greater ownership of its IoT ecosystem by deploying and managing its own devices and data infrastructure. This project will lay the foundation for the transition by designing and implementing a scalable data-ingestion platform - enabling data from diverse IoT devices (or any over data sources) to be received, processed and managed.

2. Objectives

The main objective of this project is to deploy an architecture that can:

1. Ingest data from IoT devices or any over data sources
2. Store the data into department specific repositories
3. Create a dashboard to visualize data (for admin)

3. System Overview

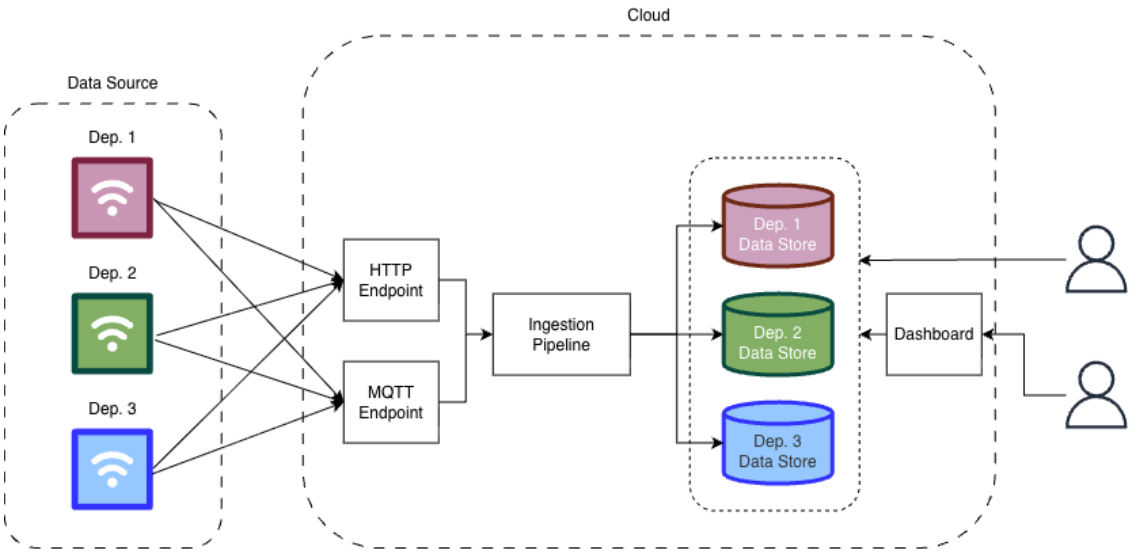


Fig. 1.1 : Data flow architecture

As shown in Figure 1.1, the data sources (or any IoT devices) interact with the data infrastructure using either HTTP requests (POST) or MQTT messages. The system must have both endpoints. This data is then passed through an Ingestion Pipeline that reads the messages, identifies the data source and the department the device belongs to, and stores the data into the correct data store.

Finally, the user should be able to query the stored data or visualize the data through a simple dashboard.

4. Project Phases

Phase 0 — Requirements & Design

1. Act as "client" stakeholder(s); students conduct requirement-gathering interviews/sessions.
2. Students produce a requirements document and translate it into a prioritized feature backlog.
3. Students propose a technology stack with a brief feasibility/trade-off analysis per component

Phase 1 — IoT Data Ingestion

1. Implement HTTP and MQTT endpoints for receiving data from physical or simulated IoT devices

2. Create a device registry containing each device's identity and department
3. Authenticate devices and validate incoming messages
4. Route valid data to the appropriate department-specific repository

Phase 2 -- Data Interaction

1. Develop a secure (users send requests with a department specific token in header) API through which authorized users can query departmental data.
2. Support basic filtering by device, department, data type, and time range.
3. Deploy a dashboarding solution, such as Grafana, and connect it to the query API or data repositories.
4. Create department-specific views displaying device status, key metrics, historical trends, and generated alerts.

5. Skills Developed

5.1 Technical

1. Design and deploy an end-to-end IoT data architecture using HTTP, MQTT, cloud services, and databases
2. Build scalable data-ingestion pipelines that validate, classify, transform, and store device data
3. Apply AI and data-analytics techniques to aggregate information, detect anomalies and generate alerts
4. Develop APIs and dashboards while applying security, monitoring, and deployment best practices.

5.2 Project management

1. Requirements elicitation and translation into features
2. Feasibility assessment and technology trade-off analysis
3. Backlog management and progress tracking against a plan
4. Root-cause analysis and structured improvement planning

6. Deliverables list

1. Requirements & architecture documentation/presentation
2. Deployed HTTP and MQTT ingestion endpoints
3. Device Registry : a key-value database data contains each device's ID and the corresponding department.
4. Automated data routing
5. Operational dashboard with department-specific views
6. Test report (functional + resilience tests)

7. Evaluation Criteria

1. Requirements analysis (15%)
2. Architecture & technology choice justification (15%)
3. Pipeline implementation (40%)
4. Document with all deliverables above mentioned (15%)
5. Testing rigor (15%)

Project 2: Real-Time IoT Device Monitoring & Cloud Dashboarding Platform

Proposed by: https://www.sdataway.com/

Key contacts: Nabil Abdennadher, Raoul Dupuis, Nizar Bouchedakh (SDATAWAY)

1. Context and Motivation

Modern connected devices (embedded systems, edge gateways, industrial sensors) require continuous monitoring of their operational health: uptime, temperature, power consumption, memory usage, and error states. Organizations increasingly need lightweight, cloud-native pipelines to collect this telemetry in real time and turn it into actionable insight through dashboards and analytics.

This project asks students to design and implement an end-to-end IoT monitoring Cloud based solution, from a simulated embedded device generating telemetry, through an edge data collector, into a cloud time-series backend, and finally into a visualization and analysis layer. Beyond the technical build, the project is structured to develop project management skills: gathering and formalizing requirements, translating them into a feature backlog, assessing feasibility, and tracking implementation against a plan.

2. Objectives

1. Design and implement a data pipeline that pushes real-time device monitoring data to the cloud.
2. Design and implement a dashboarding/analytics solution to visualize and monitor this data.
3. Practice requirements engineering and project management: capturing client needs, converting them into features, estimating feasibility/effort, and tracking delivery.
4. (Stretch goal) Run an improvement loop: identify issues/limitations in the first implementation and propose/implement a remediation plan.

3. System Overview

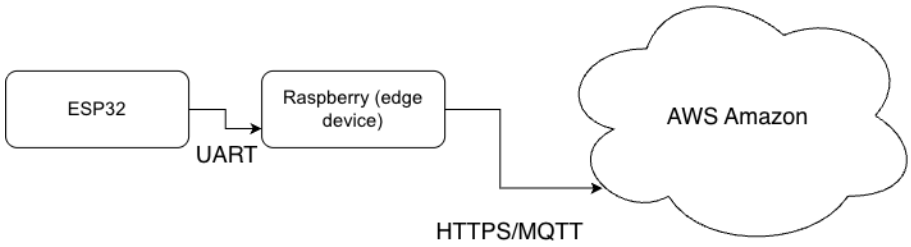


Fig 1.2: End-to-end IoT monitoring Cloud based solution

3.1 Simulated Device (Data Generator)

As shown in Fig.1.2, a Raspberry Pi or ESP32 dev board runs firmware/script that generates simulated monitoring data and streams it over a serial (UART/USB) connection. Suggested data fields:

Field	Type	Notes
Timestamp` (device-local, optional)	int/string	Device may or may not have accurate time
device_id	string	Identifies the simulated unit
uptime_seconds	int	Monotonically increasing, resets on simulated reboot
temperature_c	float	Simulated with noise/drift, occasional spikes
power_consumption_mw	float	Simulated load variation
heap_free_bytes	int	Simulated memory usage/fragmentation
device_status	Enum	e.g. `OK`, `DEGRADED`, `ERROR`, `REBOOTING`
error_code	int/string	Populated when status indicates an error

Data should be emitted at a configurable interval (e.g. every 1–5 seconds) in a well-defined serial format (e.g. JSON lines, or CSV with a defined schema).

3.2 Edge Collector (Raspberry Pi, in Fig. 1.2)

1. Acts as the gateway between the device and the cloud:
2. Reads and parses the serial stream.
3. Validates/cleans malformed data.
4. Adds a reliable timestamp (edge-side, NTP-synced) and any needed metadata (edge/gateway ID).
5. Buffers data locally if connectivity is lost (store-and-forward).
6. Pushes formatted data to AWS over the internet (protocol choice — HTTPS/REST, MQTT, or AWS IoT Core — is part of the students' technology evaluation).

3.3 Cloud Ingestion & Storage (AWS Amazon in Fig. 1.2)

1. An ingestion endpoint (e.g. AWS IoT Core, API Gateway + Lambda, or Kinesis) receives data from the edge device.
2. Data is persisted in a time-series-oriented store. Candidates to evaluate: Amazon Timestream, InfluxDB (self-hosted or InfluxDB Cloud), TimescaleDB on RDS, or DynamoDB with time-based keys.
3. Students should document the tradeoffs (cost, managed vs self-hosted, query capabilities, retention/downsampling) that justified their choice.

3.4 Dashboarding & Analysis

A visualization layer to monitor device health and explore historical trends. Candidates to evaluate: Grafana (self-hosted or Amazon Managed Grafana), Amazon QuickSight, or a custom web dashboard (e.g. React + charting library reading from an API).

Expected capabilities:

1. Live/near-real-time view of current device status per device.
2. Historical trends (temperature, power, heap usage over time).
3. Alerting/flagging on error states or abnormal readings (stretch goal).
4. Multi-device view if multiple simulated devices are run in parallel.

4. Project Phases

4.1 Phase 0 — Requirements & Design

1. Act as "client" stakeholder(s); students conduct requirement-gathering interviews/sessions.

2. Students produce a requirements document and translate it into a prioritized feature backlog.
3. Students propose a technology stack with a brief feasibility/trade-off analysis per component (device-to-edge protocol, edge-to-cloud protocol, storage, dashboard).

4.2 Phase 1 — Real-Time Data Pipeline (Device → Edge → Cloud)

1. Implement the simulated data generator (ESP32/Raspberry Pi).
2. Implement the edge collector (serial ingestion, timestamping, formatting, store-and-forward buffering).
3. Implement cloud ingestion and time-series storage.

4.3 Phase 2 — Dashboard & Data Analysis

1. Implement the dashboard against the time-series store.
2. Add basic analysis: trends, thresholds, simple anomaly flags.

4.4 Phase 3 (Stretch) — Enhancement Loop

1. Students review their own system: identify limitations (e.g. data loss scenarios, security gaps, scaling limits, dashboard usability).
2. Produce a short improvement plan with prioritized fixes.
3. Implement a subset of the improvements, time permitting.

5. Skills Developed

5.1 Technical

1. Embedded/edge programming (serial communication, data formatting)
2. Cloud architecture (IoT ingestion, time-series storage, security/auth for device-to-cloud communication)
3. Data visualization and basic analytics
4. System testing (functional, resilience/reconnection, load)

5.2 Project management

1. Requirements elicitation and translation into features
2. Feasibility assessment and technology trade-off analysis
3. Backlog management and progress tracking against a plan

4. Root-cause analysis and structured improvement planning

6. Deliverables

1. Requirements & architecture documentation/presentation
2. Working device data generator (ESP32 or Raspberry Pi)
3. Cloud ingestion + time-series storage, deployed and documented
4. Dashboard with live + historical views
5. Test report (functional + resilience tests)

7. Evaluation Criteria

1. Requirements analysis & documentation quality: 15%
2. Architecture & technology choice justification : 20%
3. Pipeline implementation (device → edge → cloud): 35%
4. Dashboard & analysis implementation: 20%
5. Testing rigor: 10%

8. Open Questions for Review

1. Should security (device auth, encryption in transit) be a graded requirement or optional enhancement?

Project 3: Cloud-Native Trade Anomaly Detection & Reporting Platform

Proposed by: https://www.dnext.io/

Key contacts: Nabil Abdennadher, Omar Bouaziz, Sami Jaballah (DNEXT)

1. Context and Motivation

Dnext is a Swiss startup based in Geneva that analyzes agricultural commodity markets. It produces daily price indices, tracks vessel movements to estimate country-level imports and exports, and develops crop models to forecast annual production. These activities rely heavily on accurate and reliable data.

We focus on assessing the reliability of trade-flow data, particularly customs data. One of the methods used is anomaly detection, which helps identify inconsistencies in reported trade flows.

Students will implement and deploy an end-to-end solution that processes customs data, detects anomalies, uses AI to explain them, and generates reports for domain experts.

2. Objectives

1. Design and implement a cloud data pipeline that ingests customs and freight data (transportation costs) into a managed data warehouse.
2. Deploy a backend service that runs the provided mirror-matching and anomaly-flagging computation at scale and exposes it through a documented API.
3. Deploy a decoupled, independently deployed dashboard that visualizes flagged anomalies.
4. Integrate a generative AI layer that turns a single flagged anomaly into a structured, narrative report.

3. System Overview

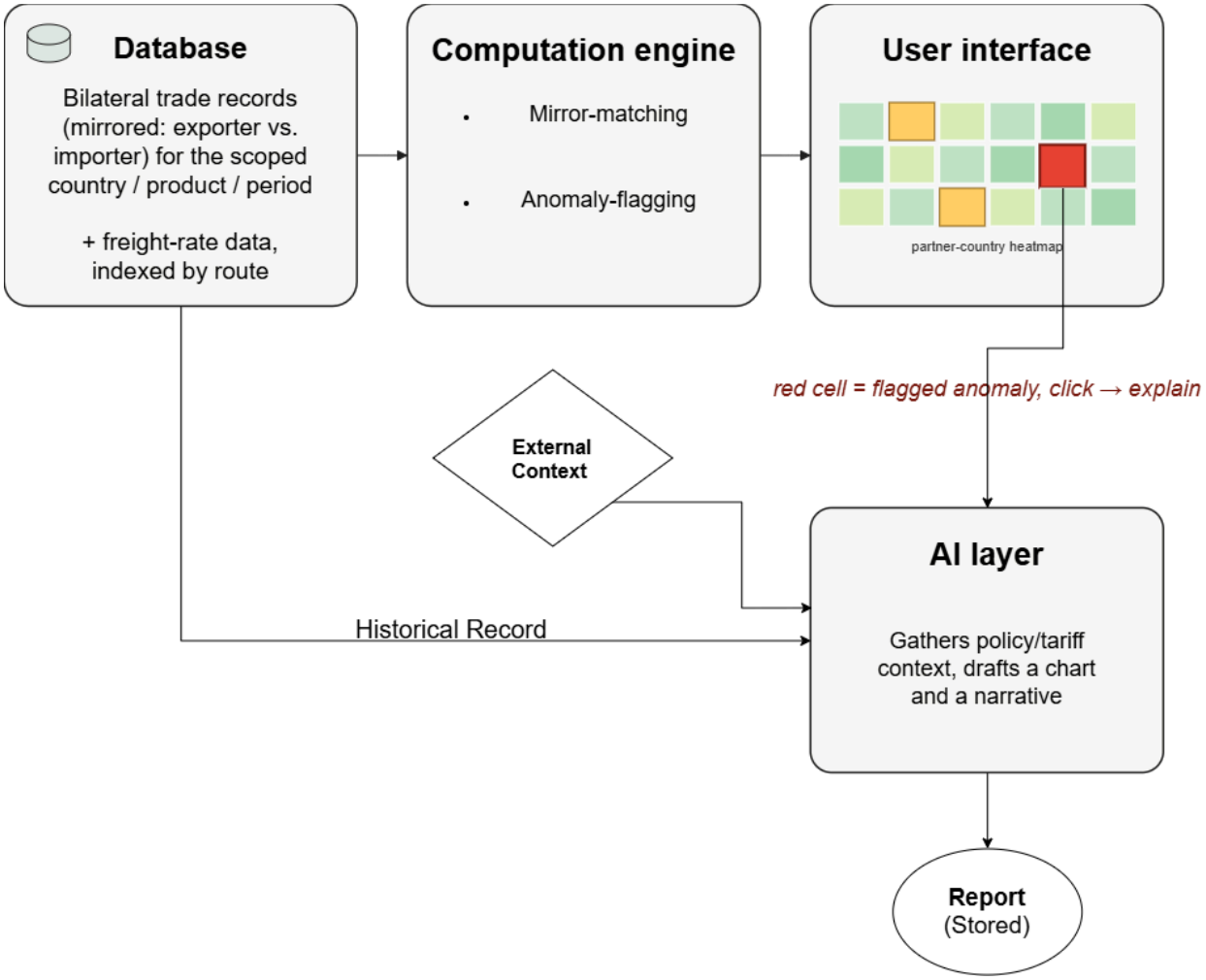


Fig. 1.3: Architecture of the anomaly detection module

4. Project Phases

Phase 0 — Requirements & Design

1. Act as "client" stakeholder(s); students conduct requirement-gathering interviews/sessions.
2. Students produce a requirements document and translate it into a prioritized feature backlog.
3. Students propose a technology stack with a brief feasibility/trade-off analysis per component

Phase 1 — Data Ingestion

1. Provision a data warehouse (e.g., Snowflake, BigQuery, Redshift, or another option the team justifies).

2. Design the schema for storing bilateral trade records (mirrored: exporter vs. importer) and freight-rate data.

Phase 2 — Deploying the Computation Engine

1. The mirror-matching and anomaly-flagging logic — concepts and implementation — is provided; the team's task is to package it and deploy it as a computation engine, choosing and justifying how and where it runs.

2. Expose the computation through a documented API.

Phase 3 — Deploying the User Interface

1. Implement a decoupled user interface (e.g., React, Vue, or another framework the team justifies), hosted independently from the computation engine.

2. Build a configuration panel where the user sets the scope: reporting country, product (HS code), time frequency, and thresholds.

3. Build a heatmap view of the top 10 partner countries against time periods, with flagged anomalies highlighted (e.g., in red) and clickable to trigger an explanation.

Phase 4 — AI-Powered Explanation & Reporting Layer

1. Integrate an LLM into a pipeline that, given a flagged anomaly, gathers supporting context and writes a narrative explanation, orchestrated as an asynchronous multi-step job rather than a single blocking call.

2. Generate a downloadable PDF report from the pipeline's output and surface it in the user interface when a user clicks a flagged cell.

3. Persist generated reports so they can be retrieved later; the storage approach (where, in what format, for how long) is left to the team to design and justify.

5. Skills Developed

1. Data warehousing: schema design and orchestrated ingestion pipelines (e.g., Snowflake, BigQuery, Redshift)

2. Cloud service deployment and integration: compute service selection (serverless vs. containers), API design, and independently deployed application layers

3. Applied generative AI engineering: prompt design, tool use, and asynchronous pipeline orchestration

6. Deliverables list

1. Requirements & architecture documentation/presentation
2. Deployed data warehouse and ingestion pipeline, documented (Phase 1)
3. Deployed backend API service running the provided computation, with documented compute and deployment choices (Phase 2)
4. Deployed decoupled frontend dashboard and AI-powered reporting pipeline, with example generated reports and their storage approach documented (Phase 3+4)
5. Test report (functional + resilience tests)

7. Evaluation Criteria

6. Requirements analysis & documentation quality (15%)
7. Architecture & technology choice justification (30%)
8. Pipeline implementation (30%)
9. Dashboard & AI-powered reporting implementation (15%)
10. Testing rigor (10%)

Project 4: Self-adaptive ML prediction of energy consumption.

Inspired for european projects: SWARM, LASAGNE and O-CEI

Key contacts: Nabil Abdennadher, Konstantin Skaburskas, Mohsen Salimi

1. Context and Motivation

The increasing integration of intermittent and decentralized renewable energy sources, together with new consumption/productions such as electric vehicles and solar panels, makes accurate prediction of energy production and consumption essential for efficient grid management. Smart meters provide detailed, real-time data that can support these forecasts. Forecasting should be performed as close as possible to where the data are generated. Processing data locally or at the edge can reduce data transfers and communication latency. Cloud is used to improve and adapt and improve forecasting accuracy.

2. Objectives

The main objective of this project is to bring the intelligence “close to the data”, such that edge devices can self-configure and react autonomously to local events. Here, the intelligence is represented by a Machine Learning Model (MLM).The deployment of the MLM on the edge devices will roughly follow a number of steps which are detailed in the next sub-section

3. System Overview

Figure 1.4 gives a schematic view of the continuous integration cycle of these MLMs. We generate an initial MLM from a set of training data. MLM is then deployed through the platform. The performance of the deployed MLM will be continuously monitored: each “time” the MLM is unable to take a reliable decision, it sends the data it failed to process to the cloud. The cloud then collects these datasets from different edge devices and triggers a new learning cycle, resulting in a new MLM which will then be pushed back to the edge devices.

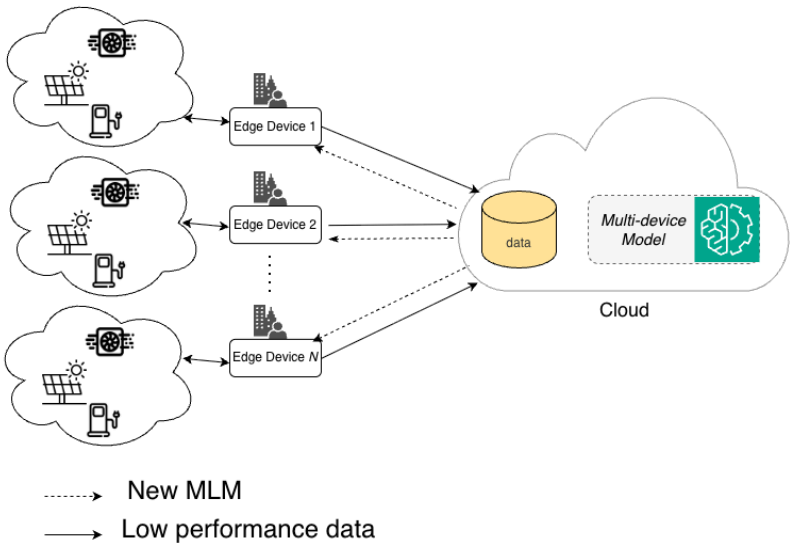


Figure 1.4: self-adaptive ML based forecasting platform for electrical energy sector

4. Project Phases

Phase 0 — Understanding and Architecture: understand the requirements of the project and provide the architecture design.

Phase 1: Analyse the data and understanding its structure (data collected from smart meters)

Phase 2: Find a suitable Machine Learning based prediction algorithm.

Phase 3: Deploy a training algorithm on a Cloud infrastructure

Phase 4: Use the Nuvla edge-to-cloud solution to deploy the whole application (MLM deployed on the edge + Training algorithm on the cloud)

Phase 5: Run and experiment/simulate the system using the data of Phase 1.

5. Skills Developed

1. ML Cloud-based services
2. Edge-to-Cloud solutions
3. State of the art of edge-to-cloud synchronisation tools

6. Deliverables list

1. Requirements & architecture documentation/presentation
2. A forecasting algorithm deployed on an edge device (could be represented by a VM)

3. A training algorithm deployed on a Cloud infrastructure
4. An integrated application (forecasting + training) deployed on an edge-to-toCloud solution and orchestrated by Nuvla.
5. Test report (functional + resilience tests)

7. Evaluation Criteria

1. Requirements analysis & documentation quality (15%)
2. Architecture & technology choice justification (30%)
3. Pipeline implementation (30%)
4. Testing rigor (15%)

Project 5: Monitoring Kubernetes clusters with AI

Inspired from a project carried out at HUG (Hôpitaux Universitaires de Genève), In particular: https://medium.com/@houcembenmahfoudh/monitor-kubernetes-clusters-with-ai-practical-use-case-7a8d5b2350b2

Key contacts: Nabil Abdennadher, Houssem Ben Mahfoudh (HUG)

1. Context and Motivation

Kubernetes environments generate large amounts of metrics, logs, and alerts that can be difficult to analyze manually. Engineers often need to use several monitoring tools and query languages such as PromQL and LogQL. This project explores how AI and Large Language Models (LLMs) can simplify Kubernetes monitoring through natural-language interaction. The goal is to help engineers investigate incidents and understand the state of their infrastructure more efficiently.

2. Objectives

The objective is to develop and deploy an AI-powered assistant for Kubernetes observability on a Cloud infrastructure. Users should be able to ask monitoring and troubleshooting questions using natural language. The system will translate these questions into queries, retrieve relevant metrics, logs and events, and provide an understandable analysis. Students will also evaluate the accuracy, reliability, security, and limitations of AI-generated responses.

3. System Overview

The functional architecture is based on a conversational AI layer connected to Kubernetes observability tools (Figure 1.5).

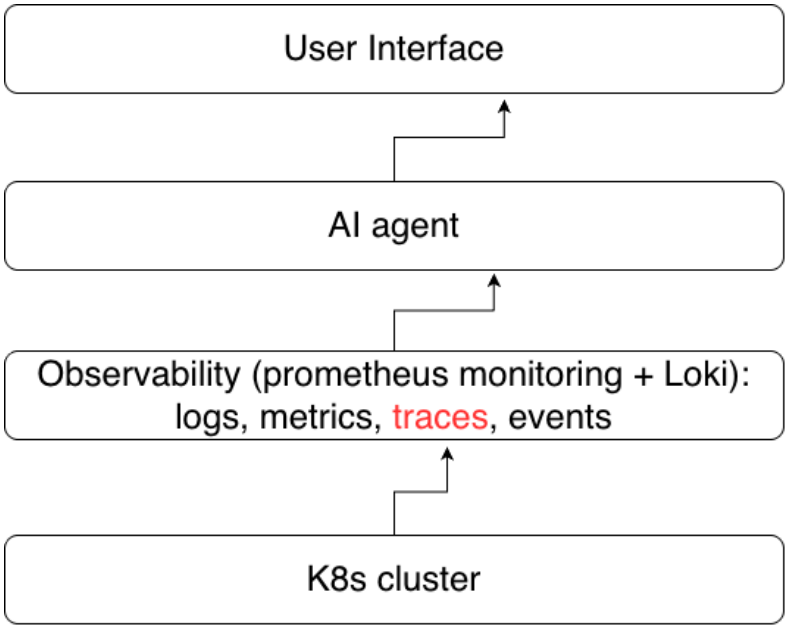


Figure 1.5 : from Natural language to operational insight (“traces” is in red, because the project will not focus on this aspect)

4. Project Phases

Phase 1 — Requirements & Design

1. Set up or use a Kubernetes cluster.
2. Set a monitoring stack (Prometheus/Mimir and Loki)
3. Collect and query metrics and logs: Student can use k8s-monitoring project based on alloy agent: https://github.com/grafana/k8s-monitoring-helm

Phase 2 — AI Platform

1. Select the LLM to use and integrate Open WebUI as the conversational front-end, wiring the agent as its backend/plugin.
2. Design and implement AI agent: integrate tool orchestration, and response structuring.
3. Implement and demo the three core use cases:
    1. cluster-wide health overview how healthy is my cluster?
    2. incident investigation (correlating alerts, metrics and logs into a timeline). Describe what is happening with … (node x , pod y, service z) ?
    3. log/metric correlation producing root-cause hypotheses. Can you make a root-cause analysis?

Phase 3 (Stretch) — Enhancement Loop

1. Students review their own system: identify limitations (e.g. security gaps, scaling limits).
2. Produce a short improvement plan with prioritized fixes.
3. Implement a subset of the improvements, time permitting.

5. Skills Developed

1. Kubernetes and infrastructure troubleshooting.
2. Observability with Prometheus/Mimir and Loki.
3. Openwebui.
4. LLM deployment and AI agents.
5. API integration and data correlation.

6. Deliverables list

1. Requirements & architecture documentation/presentation
2. Kubernetes observability environment
3. AI-powered monitoring assistant.
4. Test report (functional + resilience tests)
5. Demonstration of monitoring/troubleshooting scenarios.

7. Evaluation Criteria

1. Requirements analysis & documentation quality (15%)
2. Architecture & technology choice justification (15%)
3. Observability & AI integration (30%)
4. AI monitoring and troubleshooting capabilities (30%)
5. Testing and evaluation (10%)

8. Open Questions for Review

1. Which observability sources should be included?
2. Which local LLM offers the best accuracy/performance trade-off?

3. Should the AI only provide recommendations or eventually perform remediation?
4. How can hallucinations and incorrect conclusions be mitigated?
5. How should sensitive infrastructure data be protected?
6. How about access limits?